Universidad ORT Uruguay

Facultad de Ingeniería

Escuela de Tecnología

Obligatorio Algoritmos y estructura de datos

Emanuel Rodriguez – 306299

Grupo: N3B

Docente: Hugo Cepeda

Analista Programador

Fecha de entrega del documento 19/11/2025

Indice

Algoritmos y estructura de datos, Octubre 2025

CONTENIDOS

Introducción 3 Diseño 4 Diagrama general 4 Estructuras principales y justificación 5 Lista<Barrio> 5 ListaO<Usuario> 6 Lista<TipoBicicleta> 7 Lista<Bicicleta> 7 ListaO<Alquiler> 8 Lista<Estacion> 10 **Pruebas y resultados 12 Conclusiones 14**

Introducción

Este documento describe el diseño, implementación y pruebas del proyecto obligatorio de *Algoritmos y Estructuras de Datos*, basado en el desarrollo de un **Sistema de Bicicletas Públicas (SiBiP)**. El objetivo del sistema es permitir la gestión eficiente de estaciones, alquiler y devolución de bicicletas realizadas por los usuarios en diferentes zonas.

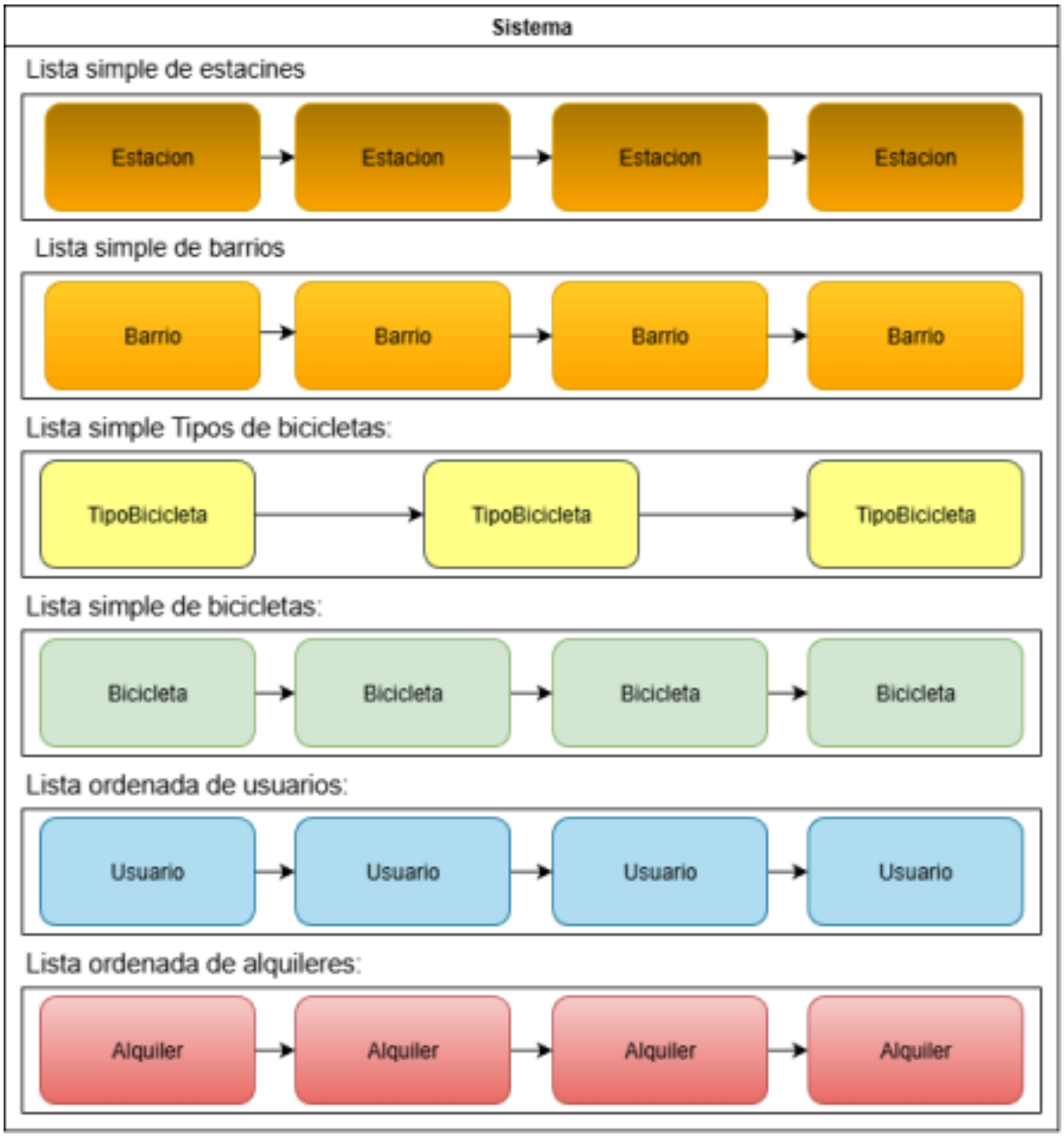
El sistema modela Estaciones, donde cada una tiene un barrio determinado asociado. Cada estación es una estructura con un número determinado de anclajes, los cuales representan los espacios físicos disponibles para estacionar bicicletas. La lógica del sistema garantiza que las operaciones de retiro y devolución respeten la capacidad máxima de cada estación, dejando en la cola de espera de cada estación aquellas que no puedan realizarse por falta de bicicletas o de espacio disponible.

Durante el desarrollo del proyecto se implementaron diversas estructuras de datos y clases para representar las entidades del sistema y gestionar las operaciones de manera eficiente. Cada decisión de diseño fue tomada en base a los requerimientos funcionales del sistema y buscando un balance entre claridad, eficiencia y mantenibilidad del código.

Finalmente, se realizaron pruebas exhaustivas para validar el comportamiento del sistema frente a distintos escenarios, incluyendo casos límite y errores esperados. Se documentan en este informe los resultados obtenidos y las conclusiones extraídas a partir del análisis de dichos resultados.

Diseño

Diagrama general



Este diagrama ilustrativo representa resumidamente nuestro sistema y lo que necesita para cumplir los requerimientos.

Sistema
Lista<Barrio> barrios
ListaO<Usuario> usuarios
Lista<TipoBicicleta> tiposBicicleta
ListaO<Alquiler> alquileresActivos
Lista<Bicicleta> bicicletas
Lista<Estacion> estaciones

Las estructuras representadas en el diagrama ilustrativo corresponden a la creación del sistema y a los componentes necesarios para su correcto funcionamiento, dando cumplimiento al requerimiento **2.1** (*Crear Sistema de Gestión*).

Estructuras principales y justificación

Lista<Barrio>

Para esta lista se definió una clase **Barrio**.

Barrio
String nombre
int bicisDisponibles
int capacidadTotalDelBarrio

Esta lista proviene de la clase *Barrio*, la cual contiene los atributos *nombre*, *bicisDisponibles* y *capacidadTotalDelBarrio*. Resulta especialmente útil para cumplir el requerimiento **3.7 (Ocupación promedio por barrio)**, ya que permite acceder directamente a estos atributos sin necesidad de realizar filtrados adicionales: basta con localizar el barrio (si existe) y leer los valores correspondientes.

El sistema garantiza que, cada vez que se registra una nueva estación, se incorpora el barrio a esta lista y se sume la capacidad de dicha estación al atributo *capacidadTotalDelBarrio*. Asimismo, en los requerimientos **2.5 (Poner bicicleta en mantenimiento)**, **2.8 (Asignar bicicleta a estación)**, **2.9 (Alquilar bicicleta)**, **2.10 (Devolver bicicleta)** y **2.11 (Deshacer últimos retiros)**, las operaciones actualizan el valor *bicisDisponibles* ya sea sumando o restando “Dependiendo del contexto” en el barrio asociado.

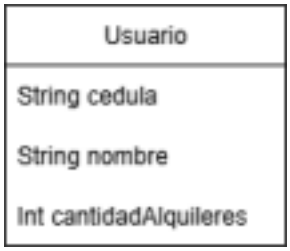
Este enfoque simplifica las búsquedas y mejora la eficiencia en los cálculos relacionados con la ocupación y disponibilidad por barrio.
En relación con el requerimiento **3.6 (Estaciones con disponibilidad mayor)**, que solicita devolver la cantidad de estaciones cuya capacidad supere un valor dado *n*, este diseño

simplifica notablemente el proceso. La búsqueda se realiza directamente sobre la lista de estaciones.

Para cumplir el requerimiento **2.7 (Eliminar estación)**. Se recorre únicamente la lista de estaciones hasta encontrar la correspondiente. Una vez identificada, se verifica que no tenga bicicletas ancladas, alquileres pendientes de devolución ni usuarios en espera de alquiler. Si todas estas condiciones se cumplen, la estación puede eliminarse de la estructura principal lo que contribuye a un diseño más simple y eficiente.

ListaO<Usuario>

ListaO representa una lista ordenada de **Usuario**.



El requerimiento **3.1** (*Obtener usuario*) nos sugiere que debemos tener una lista de usuarios para poder realizar la búsqueda por cédula.

En la clase Usuario se definen dos propiedades principales: la cédula, utilizada como clave única para identificar a cada usuario, y el nombre. Estas propiedades se deducen directamente del requerimiento **2.3** (*Registrar Usuario*), ya que son los únicos datos solicitados.

Además, se incluye la propiedad cantidad de alquileres, una decisión del equipo para facilitar el cumplimiento del requerimiento **3.10** (*Usuario con mayor cantidad de alquileres*). Gracias a esta propiedad, basta con recorrer la lista de usuarios y seleccionar aquel con el mayor valor en este contador.

> **Nota:** Para que esto funcione correctamente, es necesario incrementar el contador cada vez que un usuario realiza un alquiler.

La elección de implementar una lista ordenada en lugar de una lista simple se justifica con el requerimiento **3.2** (*Listar usuarios*), donde se solicita listar a todos los usuarios del sistema en orden alfabético ascendente por nombre. Con este diseño, los usuarios se registran directamente en el orden requerido, evitando así realizar una recorrida adicional de ordenamiento antes de mostrar los datos.

Lista<TipoBicicleta>

TipoBicicleta
String nombre int cantidadAlquileres

Esta lista simple contiene los tipos de bicicletas contemplados en el sistema: **MOUNTAIN, URBANA y ELÉCTRICA**, definidos a partir del requerimiento **2.4**.

La decisión de incluir esta lista se tomó para dar respuesta al requerimiento **3.8** (*Ranking por tipo de uso*), que solicita calcular la cantidad total de alquileres por tipo de bicicleta. Con este enfoque, el sistema mantiene actualizado el contador de alquileres por cada tipo de bicicleta, evitando recorrer todas las bicicletas o alquileres al momento de realizar el cálculo.

> **Nota:** Cada vez que se registra un alquiler, debe incrementarse el contador correspondiente al tipo de bicicleta.

Aunque en principio solo manejamos estos tres tipos de bicicletas, la solución es escalable, ya que permite incorporar fácilmente nuevos tipos en el futuro. Además, el número de tipos siempre será significativamente menor que la cantidad total de bicicletas, lo cual garantiza la eficiencia de este diseño.

Por último, esta lista también brinda un control conceptual sobre los tipos de bicicleta, funcionando como un conjunto de valores precargados que previene errores de tipado al registrar nuevas bicicletas.

Lista<Bicicleta>

Bicicleta
String codigo TipoBicicleta tipo EstadosEnum estado Estacion? estacion

Para cumplir el requerimiento **3.3** (*Listar bicicletas en depósito*), esta lista almacena únicamente las bicicletas en estado **“Disponible”** o **“Mantenimiento”**. Al registrar nuevas bicicletas, estas deben agregarse al final de la lista.

Las bicicletas en estos estados no poseen una estación asignada y, por lo tanto, no forman parte de ninguna lista ordenada de bicicletas dentro de una estación.
Basándonos en el punto **2.5** (*Poner bicicleta en mantenimiento*), cuando realicemos esta acción si la bici tiene una estación asignada, se elimina la estación, se agrega a esta Lista<Bicicleta> perteneciente a Sistema y pasa a estado **Mantenimiento**. Si ya está en depósito

(Lista<Bicicleta>) solo cambia su estado.

Para cumplir el requerimiento **2.6** (*Reparar bicicleta*), simplemente buscamos la bicicleta en Lista<Bicicleta> y le cambiamos su estado a **Disponible.**

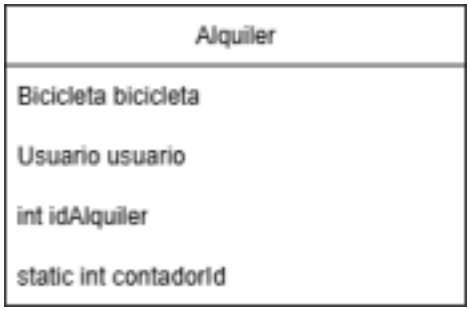
El campo estación de una bicicleta se mantiene inalterado mientras está en alquiler. Solo se actualiza cuando la bicicleta es devuelta en una estación distinta, o bien cuando se traslada a depósito (Lista<Bicicleta>).

Según lo definido en el requerimiento **2.4** (*Registrar Bicicleta*), cada bicicleta posee dos atributos principales:

- **Código** (único, de 6 caracteres).
- **Tipo de bicicleta**, descrito en la lista `Lista<TipoBicicleta>`.

Finalmente, para evitar errores de tipeo al asignar estados (por ejemplo: Disponible, Alquilada o Mantenimiento), se implementó un enumerado de estados que garantiza consistencia y control en la gestión de las bicicletas.

ListaO<Alquiler>



Esta lista ordenada representa los alquileres de bicicletas activos. Para gestionarlos, se implementó la clase Alquiler, que almacena:

- La bicicleta alquilada, en su estado correspondiente.
- El usuario que realiza el alquiler.

La lista se encuentra ordenada por id en forma descendente, lo que permite que el último alquiler registrado quede siempre en la primera posición de la lista.
El requerimiento **2.11** (*Deshacer últimos retiros*) indica que deben revertirse los últimos *n* alquileres, devolviendo las bicicletas a su estación de origen y marcándolas como disponibles. Gracias al orden descendente de la lista, basta con recorrer desde el inicio las *n* posiciones necesarias, actualizar el estado de las bicicletas a disponible y, al alcanzar *n*, redefinir el nodo correspondiente como nuevo inicio de la lista.

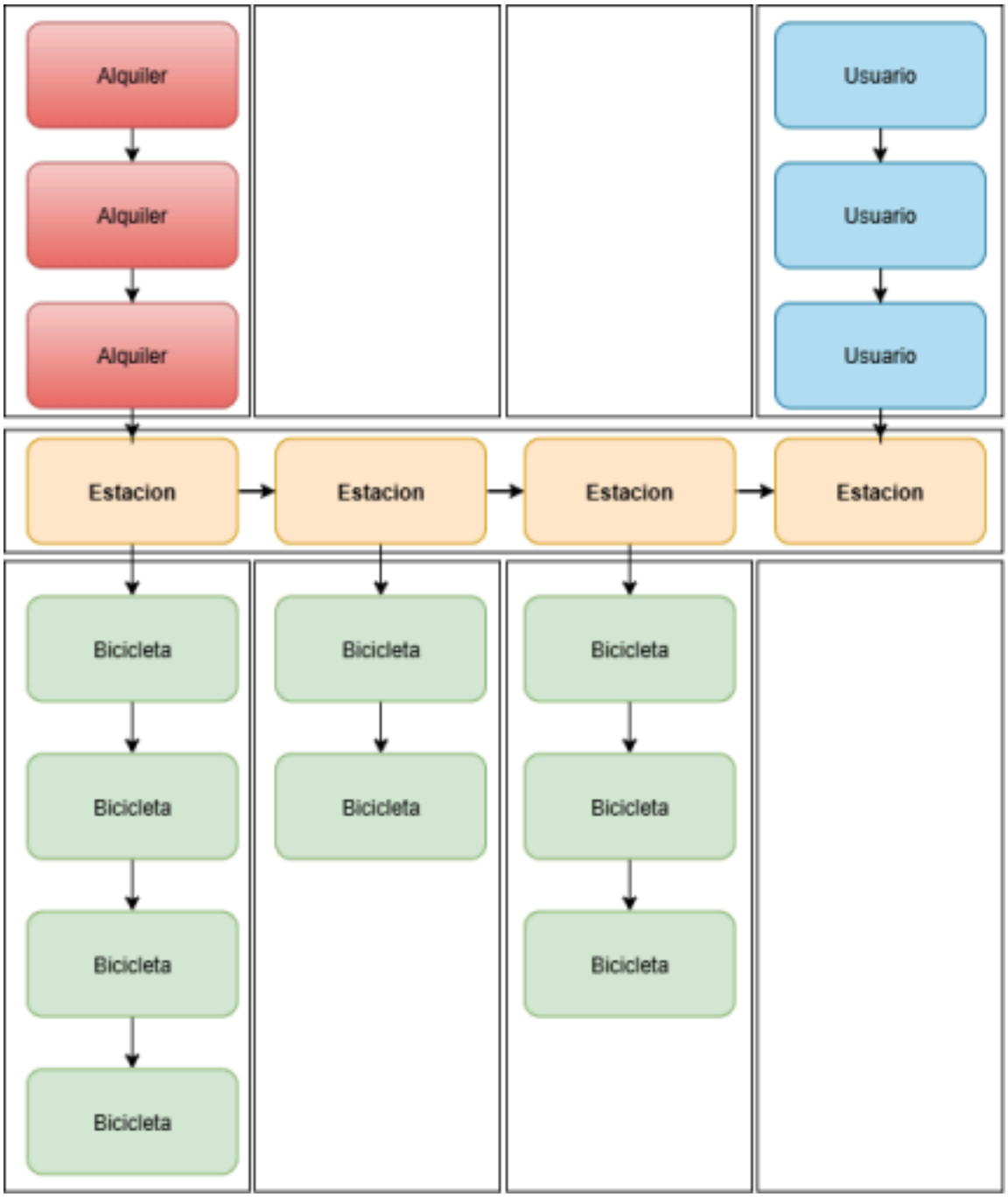
Como cada bicicleta conoce su estación de origen mediante la propiedad implementada en `Lista<Bicicleta>`, no es necesario asignar manualmente la estación en este proceso.

Asimismo, la lista cumple un rol clave en el requerimiento **2.9** (*Alquilar bicicleta*): al momento de registrar un alquiler, se crea un objeto Alquiler, se cambia el estado de la bicicleta a alquilada y se inserta en la `ListaO<Alquiler>` de manera ordenada.

Por último, para dar soporte al requerimiento **2.10** (*Devolver bicicleta*), si es necesario devolver una bicicleta que no corresponde a los últimos retiros, se localiza en la lista el elemento asociado, se lo elimina y, en el mismo proceso, se actualiza el estado de la bicicleta y se asigna nuevamente a la estación correspondiente.

Lista<Estacion>

Diagrama representativo de la Lista de estaciones:


Alquiler
Alquiler
Alquiler
Usuario
Usuario
Usuario
Estacion
Estacion
Estacion
Estacion
Bicicleta
Bicicleta
Bicicleta
Bicicleta
Bicicleta
Bicicleta
Bicicleta
Bicicleta
Bicicleta

Estacion
String nombre
Barrio barrio
int capacidad
int contadorBiciDisponible
ListaO<Bicicleta> Bicis
Cola<Usuario> UesperandoAlquilar
Cola<Alquiler> UesperandoEntregar

La Estación está asociada a un barrio.
El contadorBiciDisponible representa la cantidad de bicis disponibles en simultáneo en la estación.
La ListaO<Bicicleta> representa las bicis disponibles de la estación.

Requerimiento 2.2 (Registrar Estación):
El método solicita nombre, barrio y capacidad. La capacidad de bicicletas debe ser mayor a 0. Se verifica que la estación no exista, se obtiene el barrio y se le asigna a la estación. Si no existe se crea y es asignado a la estación.

Requerimiento 3.5 (Listar bicis de estación):
Cada estación implementa una lista ordenada de bicicletas, ordenada por código de forma creciente. Esto permite devolver directamente la lista en el formato solicitado, evitando un paso adicional de ordenamiento.

Requerimiento 2.8 (Asignar bicicleta a estación):
La lista de bicicletas de una estación no puede superar su capacidad. El flujo definido es: 1. Al registrarse una bicicleta, ésta entra en la `Lista<Bicicleta>` general del sistema con estado “Disponible”.

2. Para asignarla a una estación, se verifica que la `ListaO<Bicicleta>` de esa estación no haya alcanzado su capacidad.

3. Si la condición se cumple, la bicicleta se elimina de la lista general del sistema y se agrega a la lista de la estación.

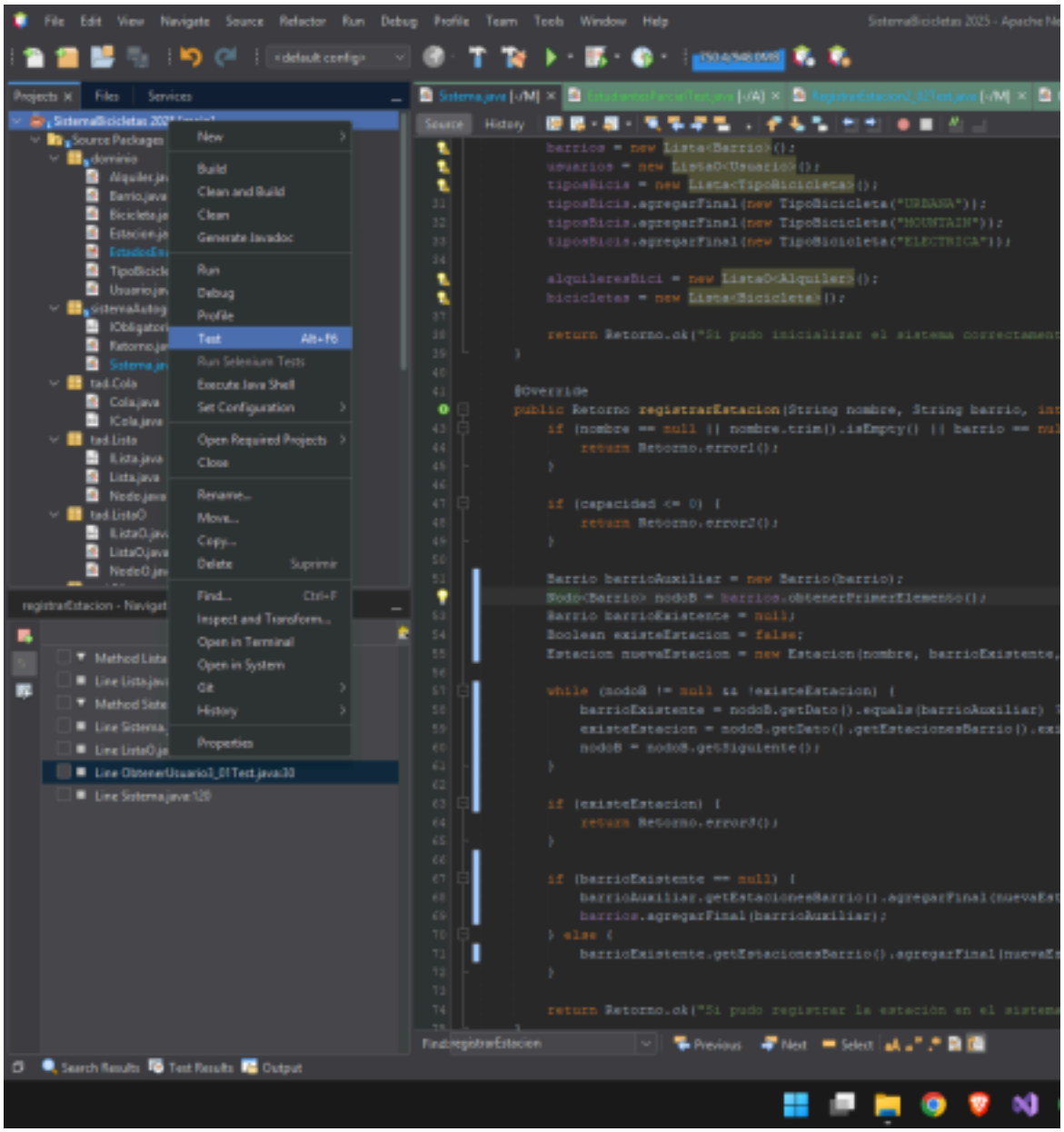
Requerimientos 2.9 (Alquilar bicicleta) y 3.9 (Usuarios en espera por alquiler): Se implementó una Cola<Usuario>, que representa a los usuarios en espera cuando una estación no tiene bicicletas disponibles. Se eligió una cola en lugar de una lista, ya que el primer usuario en ingresar debe ser el primero en obtener una bicicleta (FIFO).

Requerimiento 2.10 (Devolver bicicleta):
Se implementó también una Cola<Alquiler> para manejar devoluciones. Si una bicicleta no puede ser devuelta porque la estación está llena, queda en espera en esta cola. El criterio FIFO asegura que el primer alquiler en ingresar será el primero en poder devolver la bicicleta una vez liberado un lugar.

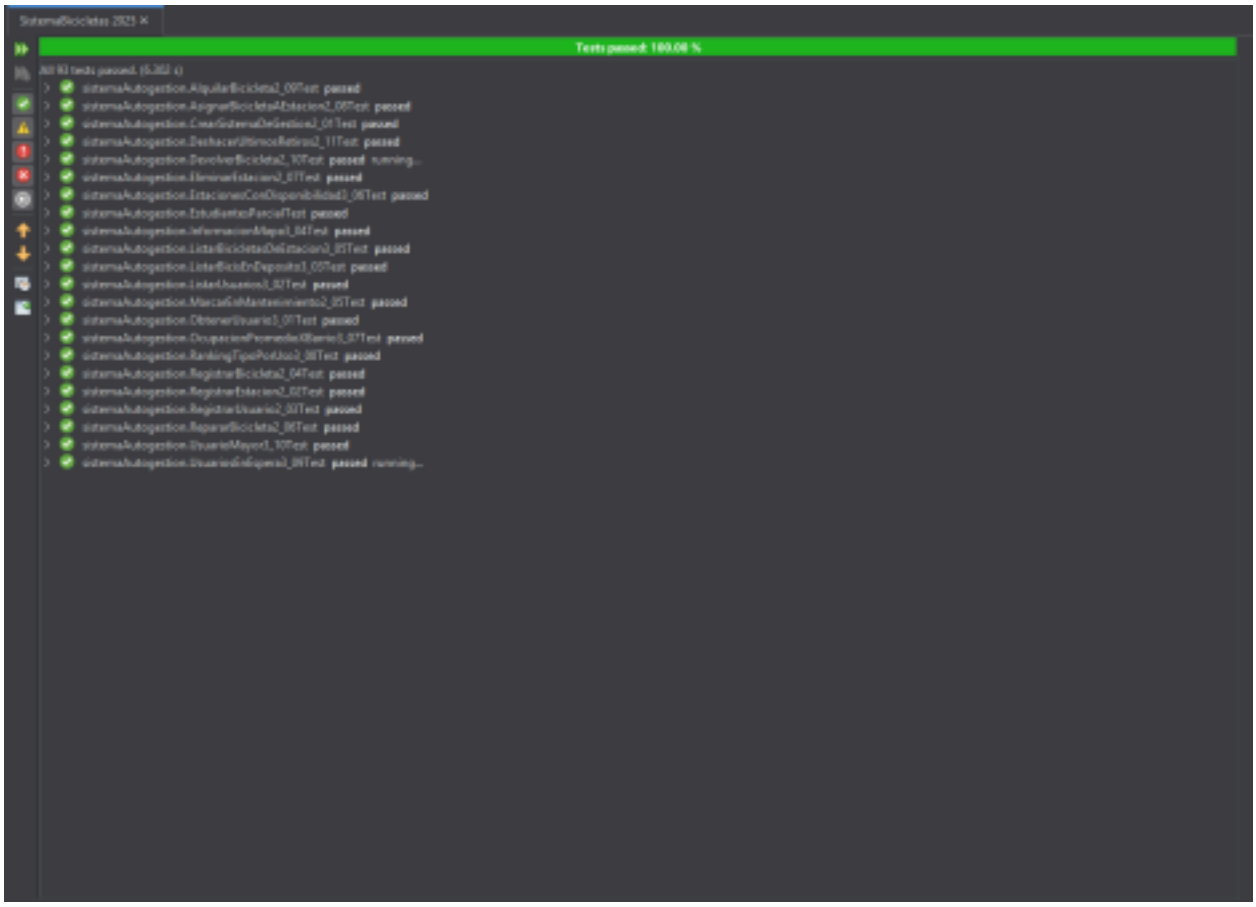
Pruebas y resultados

En esta sección se incluyen los casos de prueba, resultados esperados y obtenidos. También se adjuntan capturas de pantalla que evidencian el correcto funcionamiento.

Para correr los test, se selecciona click derecho en el proyecto y se selecciona la opción “Test” o alt + F6.



La siguiente captura muestra los resultados a partir de correr todos los test, incluido el set entregado por el profesor, archivo el cual nombramos “EstudiantesParcialTest.java”.



Conclusiones

A lo largo del desarrollo del proyecto del Sistema de Bicicletas Públicas (SiBiP), se logró modelar de manera efectiva las entidades principales del sistema —estaciones, bicicletas y usuarios— respetando las restricciones y requerimientos establecidos. Además de estas entidades, se agregaron algunas más —Barrio, TipoBicicleta y Alquiler— para cumplir de mejor manera con los requerimientos.

Se utilizaron estructuras de datos adecuadas para asegurar un manejo eficiente de la información.

El sistema fue probado bajo diferentes escenarios, incluyendo casos normales y situaciones límite, lo que permitió verificar su correcto funcionamiento y robustez. Las decisiones de diseño tomadas durante la implementación contribuyeron a una solución eficiente basada en el contenido del curso.